Lab 4: Simple Python Database Connections

Lab4_Solution

Terminal identity	amnajaved-dev1@ubuntu
GitHub username	amnajaved-dev1
Databases fork URL	https://github.com/amnajaved-dev1/Databases
Submission directory	DevOps/Labs/Lab04

Summary of results. One file, `db_connect.py`, was edited through four stages: MySQL with values in code, MongoDB with values in code, MongoDB with `os.getenv()`, and MongoDB with a credentials file read by `configparser`. Python ran directly on Ubuntu; Docker ran only the databases. The final file is the credentials-file version.

Task 1: Fork and Clone the Databases Repository

I forked the instructor's Databases repository and kept the name. The page shows **amnajaved-dev1/Databases** and states it was forked from WaqasSaleem97/Databases. Fork URL: https://github.com/amnajaved-dev1/Databases

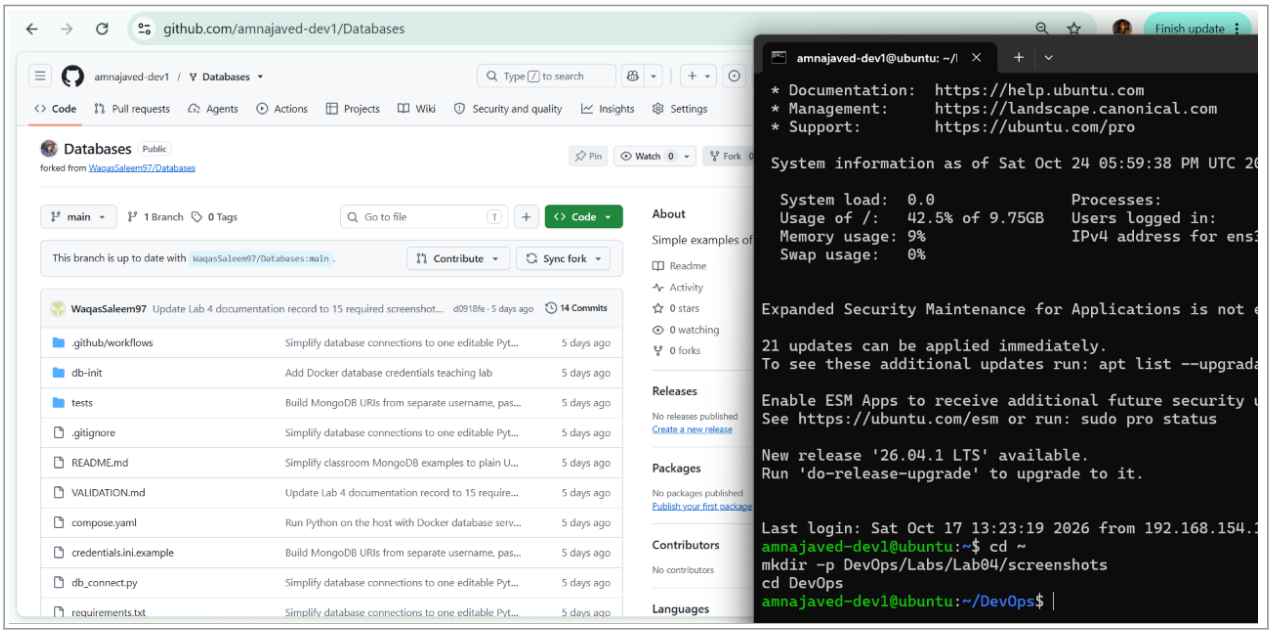


Figure 1: databases_fork.png - fork page with the Ubuntu terminal in split screen.

I then cloned my own fork over SSH into `~/Databases`.

```
cd DevOps
amnajaved-dev1@ubuntu:~/DevOps$ cd ~
git clone git@github.com:amnajaved-dev1/Databases.git
cd Databases
Cloning into 'Databases'...
remote: Enumerating objects: 101, done.
remote: Counting objects: 100% (101/101), done.
remote: Compressing objects: 100% (90/90), done.
remote: Total 101 (delta 30), reused 1 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (101/101), 39.86 KiB | 82.00 KiB/s, done.
Resolving deltas: 100% (30/30), done.
amnajaved-dev1@ubuntu:~/Databases$
```

Figure 2: databases_clone.png - git clone of my fork completed.

Task 2: Install Docker, Docker Compose, and Python

After installing docker.io and curl, adding the Compose plug-in, adding my account to the docker group and reconnecting, all three tools report versions: Git 2.43.0, Docker 29.1.3, and Docker Compose v5.6.1. Python 3, venv and pip were installed and a `.venv` was created in ~/Databases.

```
amnajaved-dev1@ubuntu: ~/I
  0:06:46 92 31.1M   92 28.7M    0     0  73982      0  0:07:21  0:06:47 92 31.1M   92 28.8M    0     0  73994      0
  0:07:21  0:06:48 92 31.1M   92 28.8M    0     0  73885      0  0:07:22  0:06:49 92 31.1M   92 28.9M    0     0  739
20      0  0:07:22  0:06:50 93 31.1M   93 29.0M    0     0  74020      0  0:07:21  0:06:51 93 31.1M   93 29.1M    0
   0  74196      0  0:07:20  0:06:52 94 31.1M   94 29.2M    0     0  74333      0  0:07:19  0:06:53 94 31.1M   94 29.
4M    0     0  74472      0  0:07:18  0:06:54 94 31.1M   94 29.5M    0     0  74529      0  0:07:18  0:06:55 95 31.1M
   95 29.6M    0     0  74781      0  0:07:16  0:06:56 95 31.1M   95 29.7M    0     0  74871      0  0:07:16  0:06:57
 96 31.1M   96 29.9M    0     0  75170      0  0:07:14  0:06:58 96 31.1M   96 30.0M    0     0  75225      0  0:07:14
  0:06:59 96 31.1M   96 30.1M    0     0  75162      0  0:07:14  0:07:00 96 31.1M   96 30.1M    0     0  75139      0
  0:07:14  0:07:01 97 31.1M   97 30.3M    0     0  75343      0  0:07:13  0:07:02 97 31.1M   97 30.4M    0     0  753
13      0  0:07:13  0:07:03 97 31.1M   97 30.5M    0     0  75459      0  0:07:13  0:07:04 98 31.1M   98 30.6M    0
   0  75472      0  0:07:12  0:07:05 98 31.1M   98 30.7M    0     0  75560      0  0:07:12  0:07:06 98 31.1M   98 30.
8M    0     0  75610      0  0:07:12  0:07:07 99 31.1M   99 30.9M    0     0  75824      0  0:07:10  0:07:08 99 31.1M
   99 31.1M    0     0  76027      0  0:07:09  0:07:09100 31.1M  100 31.1M    0     0  76005      0  0:07:09  0:07:09
 --:--:--  121k
amnajaved-dev1@ubuntu:~/Databases$ git --version
docker --version
docker compose version
git version 2.43.0
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
Docker Compose version v5.6.1
amnajaved-dev1@ubuntu:~/Databases$
```

Figure 3: docker_installation.png - git, docker and docker compose versions.

Task 3: Start the Database Services

`docker compose up -d --wait` started mysql, mongo-dev, mongo-test and mongo-prod. All four containers are healthy.

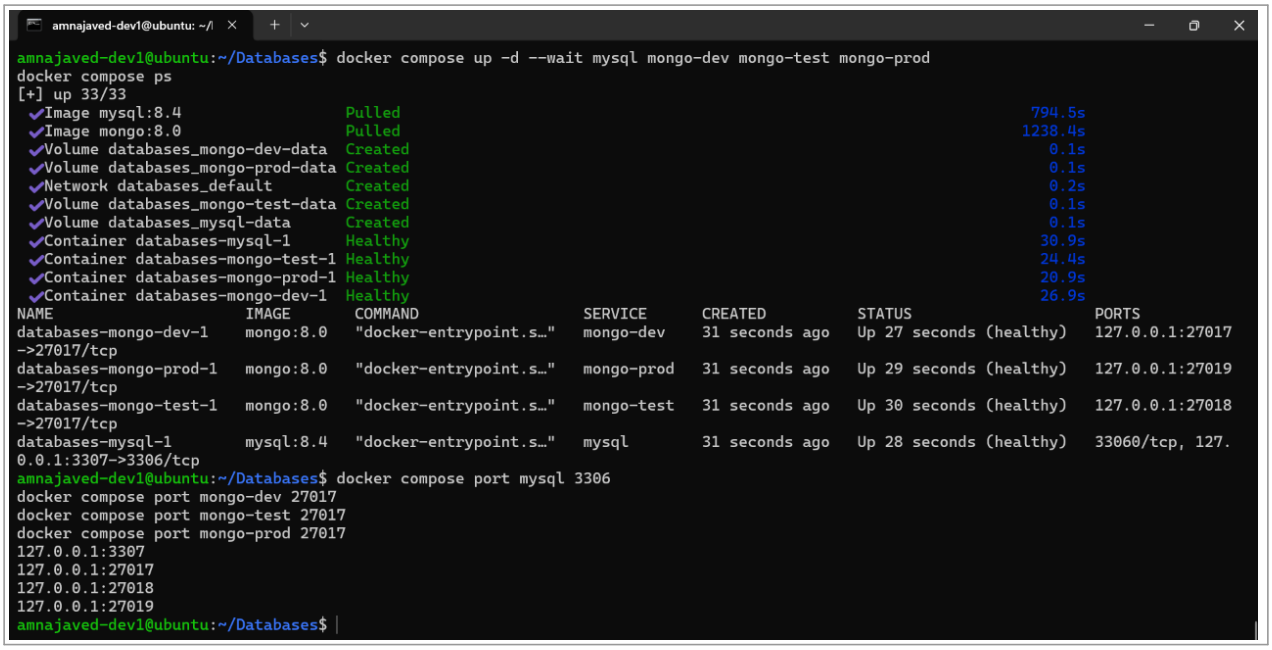


Figure 4: database_services.png - four services healthy.

The host endpoints reported by `docker compose port` are: MySQL 127.0.0.1:3307, MongoDB dev 127.0.0.1:27017, test 127.0.0.1:27018, simulated prod 127.0.0.1:27019.

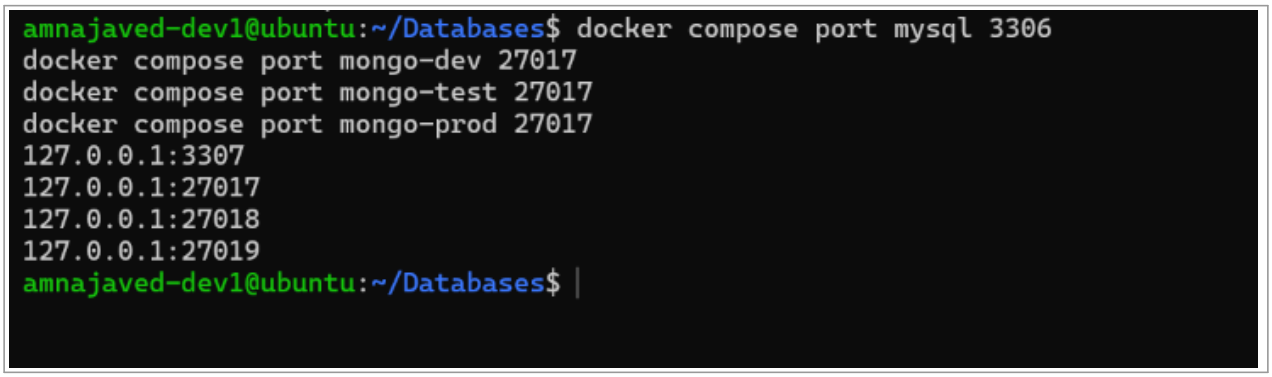


Figure 5: database_ports.png - host ports of the four databases.

Task 4: MySQL with Values in the Code

`mysql.connector.connect()` receives the host, port, user, password and database name written directly in the code. The program prints the connected database, the server host and port, and the environment value stored in the `environment_info` table. The result shows database school_mysql, server 127.0.0.1 3307 and environment dev.

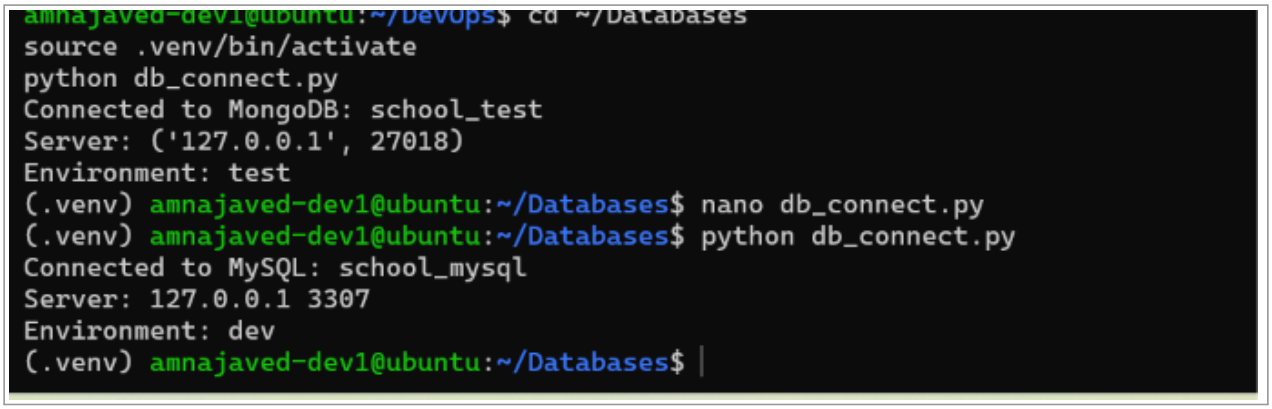


Figure 6: mysql_hardcoded_connection.png - MySQL connection with hardcoded values.

Task 5: MongoDB with Values in the Code

Username, password, endpoint and database name are held in separate variables and joined by + into the URI. `database.command("ping")` sends a real request, which proves connectivity; creating the client alone does not. The result shows database school_dev, port 27017 and environment dev.



Figure 7: mongo_hardcoded_connection.png - school_dev, 27017, environment dev.

Task 6: MongoDB with os.getenv()

The same connection block now takes its four settings from environment variables (MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT, MONGO_DATABASE). Python inherits the exported variables from the shell, so the unchanged program reaches three different databases.

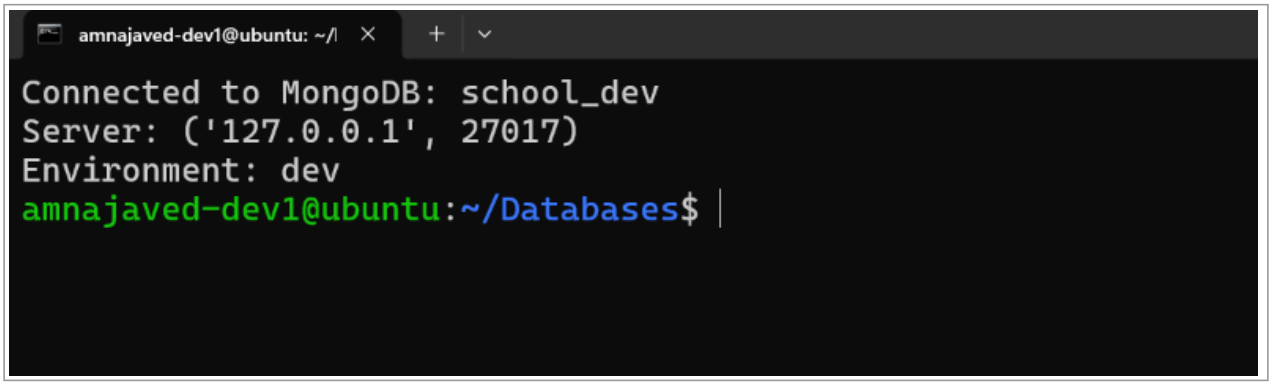


Figure 8: mongo_env_dev.png - school_dev, 27017, environment dev.

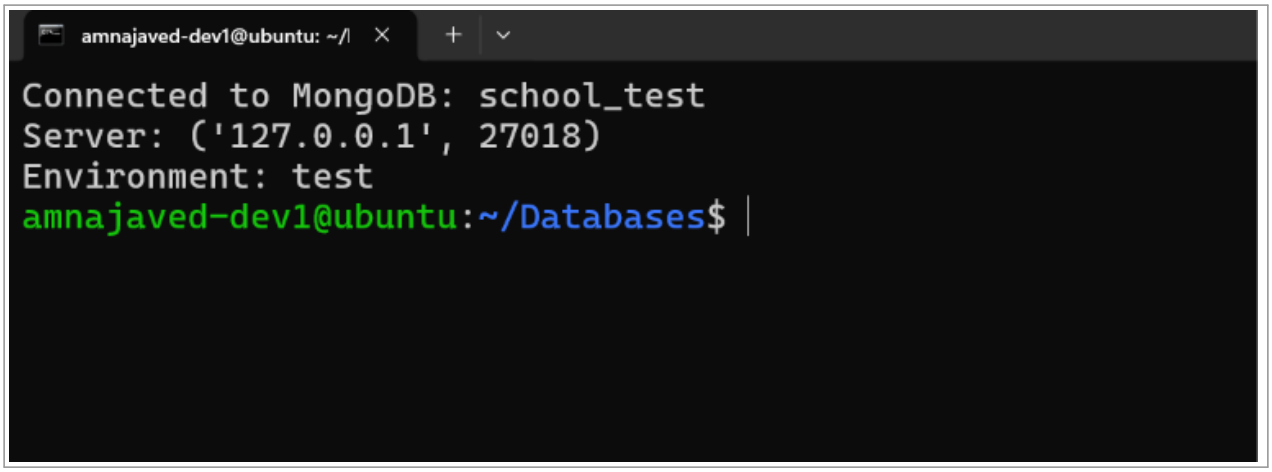


Figure 9: mongo_env_test.png - school_test, 27018, environment test.

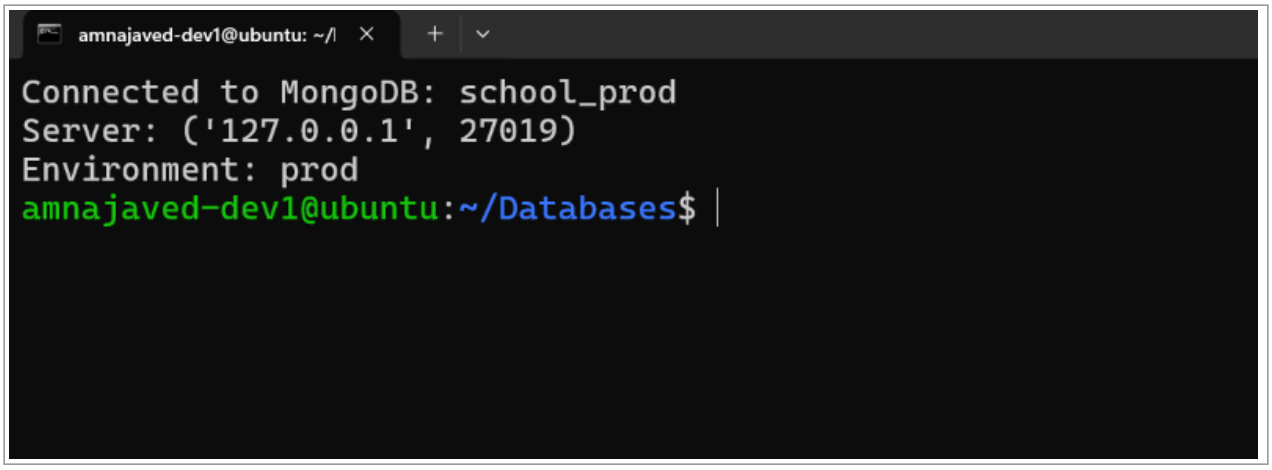


Figure 10: mongo_env_prod.png - school_prod, 27019, environment prod.

Task 7: MongoDB with a Credentials File

I copied `credentials.ini.example` to a local `credentials.ini` (ignored by Git) containing dev, test and prod sections.

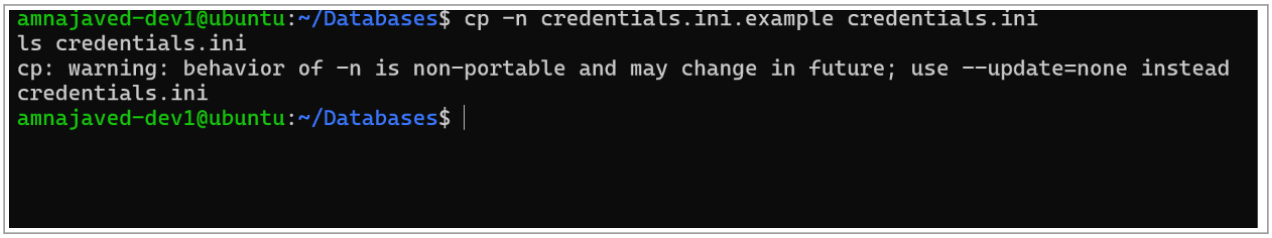


Figure 11: credential_files_created.png - credentials.ini created.

The file is restricted to my account with `chmod 600`, and `stat` confirms mode 600.

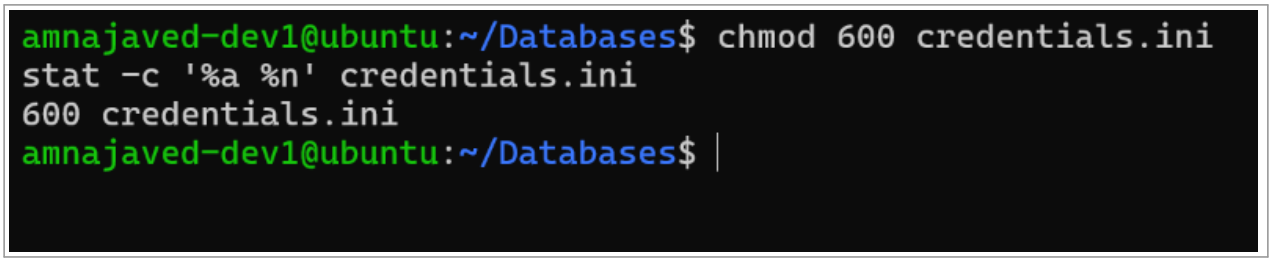


Figure 12: credential_file_permissions.png - mode 600 credentials.ini.

APP_ENV selects the INI section (default dev). The connection and output block is unchanged.

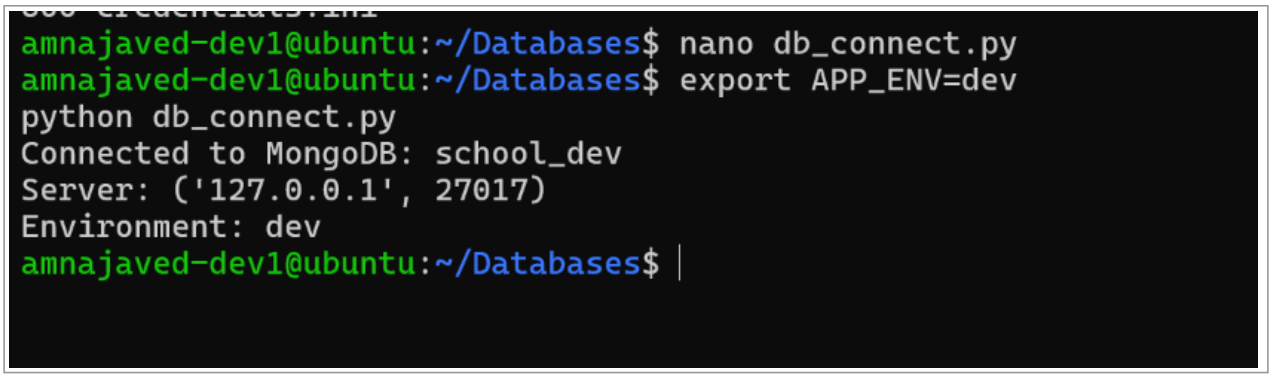


Figure 13: mongo_file_dev.png - APP_ENV=dev: school_dev, 27017, dev.

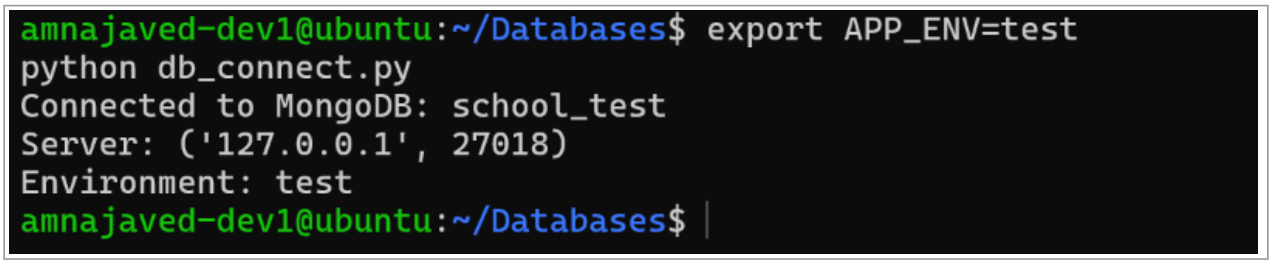


Figure 14: mongo_file_test.png - APP_ENV=test: school_test, 27018, test.

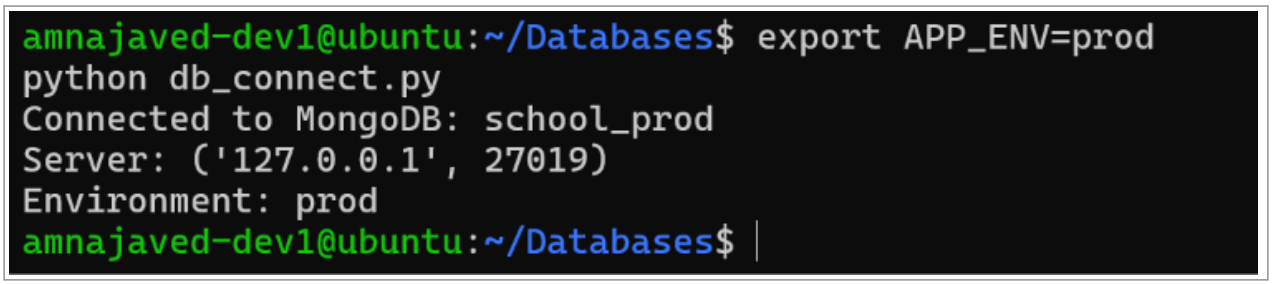


Figure 15: mongo_file_prod.png - APP_ENV=prod: school_prod, 27019, prod.

Exam Evaluation Questions

1. A Missing Environment Variable

Prediction: with MONGO_DATABASE unset, the program stops before connecting and prints the warning message.

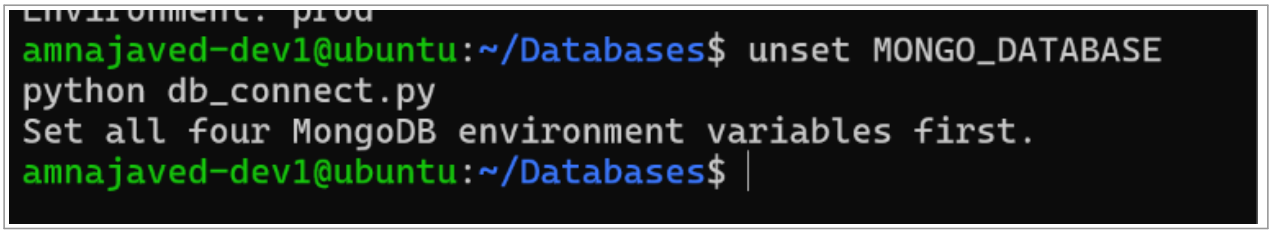


Figure 16: Q1_missing_environment.png - message after unset MONGO_DATABASE.

Explanation: `os.getenv("MONGO_DATABASE")` returned `None` because the variable no longer exists in the shell environment. The condition `not database_name` is therefore true, so the `if` statement runs `raise SystemExit(...)`. The program exits before the URI is built and before MongoClient is created. Restoring `export MONGO_DATABASE=school_prod` makes all four values present again, so the program connects.

2. Which Source Does the File Version Read?

Prediction: the file version connects to dev.

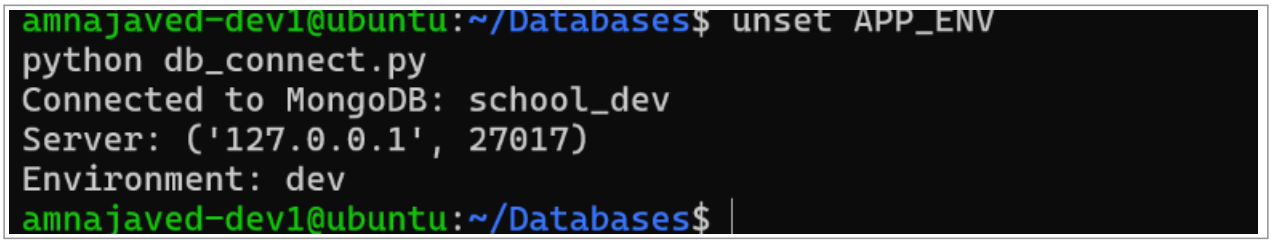


Figure 17: Q2_file_source_selection.png - dev connection after unset APP_ENV.

Explanation: the file version never reads MONGO_USERNAME, MONGO_PASSWORD, MONGO_ENDPOINT or MONGO_DATABASE. It reads only APP_ENV, and `os.getenv("APP_ENV", "dev")` falls back to **dev** when it is unset. That section name picks the settings from credentials.ini, so the leftover prod variables in the shell are ignored. Setting `APP_ENV=test` then selects the test section.

3. A Wrong Password in the File

Prediction: with a wrong password in the [test] section, MongoDB rejects the login with an authentication error.

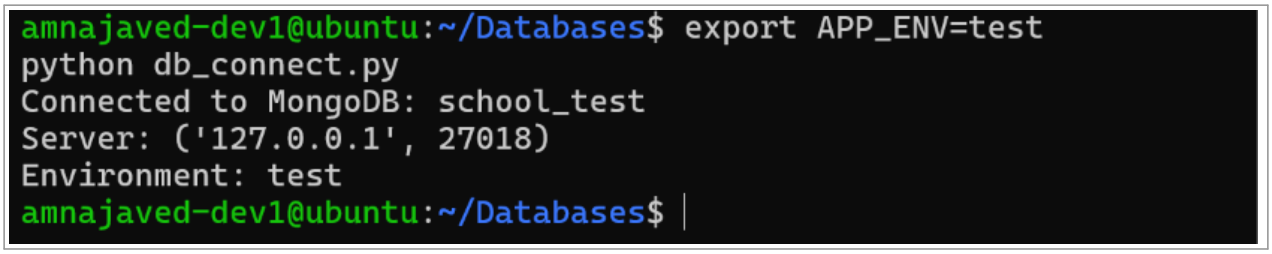


Figure 18: Q3_wrong_file_password.png - APP_ENV=test run.

Explanation: the endpoint only tells the client where the server is. The server still checks the username and password for the authentication database, so a correct endpoint and database name cannot make an incorrect password valid. After the test, the supplied dummy test password was restored, the connection was run successfully again, APP_ENV was set back to prod, and credentials.ini was not committed.

Conclusion

The connection code stayed the same while its configuration source changed from values in code, to environment variables, to a credentials file. Each target was verified using the database name, endpoint port and the environment value stored in the database. No passwords or credential-bearing URIs are shown in this document.